NMIMS, NSOMASA . MLOps LAB . PROJECT 1

Credit Risk Model with a CI/CD Quality Gate

Every change to code, configuration, features or data is tested, tracked and accepted only if it does not make the model worse, and the gate itself is justified with evidence from our own runs.

A reproducible credit-risk build with Git, DVC, Pydantic-validated configs, MLflow tracking and a GitHub Actions quality gate.

21	0.7948	3	28 / 28	8 / 8
tracked runs	baseline PR-AUC	PR scenarios	unit tests pass	tasks complete

REPORT DETAILS

Field	Detail
Assessment	Project 1, Credit Risk Model with a CI/CD Quality Gate
GitHub repo link	https://github.com/AdityaNariyapara/credit-risk-cicd-gate
Course	Elective II, Machine Learning Operations (MLOps Lab)
Submitted to	Dr. Santosh Kumar
Submitted by	Aditya Nariyapara (B009) and Shreya Dutt (B016)
Institute	NSOMASA, NMIMS
Dataset	credit_card_customers v1 (800 rows) and v2 (1,100 rows)
Tools	Python, Git and GitHub, DVC, MLflow, GitHub Actions, pytest, Pydantic
Date	October 2026

PROJECT WORKFLOW

01 Version data . 02 Configure . 03 Engineer features . 04 Train and track . 05 Gate on every PR . 06 Data-change drill . 07 Threshold and fairness . 08 Report

Table of Contents

Executive Summary
01 Problem and Objectives
02 Architecture and Tools
03 Repository, Code and Data Versioning (Task 2)
04 The Data
05 Configuration Management (Task 3)
06 Cleaning and Feature Engineering (Task 4)
07 Model Training and Tuning (Task 5)
08 Experiment Tracking, Registry and Lineage (Task 5)
09 Metric Justification: Why Not Accuracy
10 The Quality Gate and CI (Task 6)
11 Data-Change Drill, v1 versus v2 (Task 7)
12 Threshold Comparison and Fairness Defense (Task 8)
13 Lifecycle Mapping (Task 1)
14 Worked Example: Reproduce From a Fresh Clone
15 Results, Contributions and Reflection
16 Deliverables Checklist

Executive Summary

A retail bank retrains its credit-card default model every month. Recently an untested change lowered model quality and nobody noticed for two weeks, and afterwards nobody could say which code, configuration or data version had produced the faulty model. The risk team now wants a workflow in which every change is versioned, tested automatically, tracked in an experiment log, and accepted only if the model is at least as good as the current baseline, and they want the team to be able to defend why the gate threshold is the right one, not just that a gate exists.

This project delivers that workflow end to end. Data and code are versioned together, every training run is logged to MLflow with its parameters, metrics, dataset version and configuration, and a quality gate runs on every pull request. The gate trains the model with a fixed CI configuration, compares the result to a baseline stored in the repository, and blocks the merge when a test fails or the model gets worse. The dataset has about one high-risk customer in five, so we gate on average precision (PR-AUC) rather than accuracy, and we justify that choice against two alternatives using our own run history.

0.7948	0.690 to 0.869	+0.055	0.35 vs 0.20
baseline PR-AUC (gate bar)	PR-AUC across 21 runs	v2 vs v1, within noise	Low vs High FN rate

Ground rules

Every tool is free and open source. No paid cloud account is used, no secrets are stored in the repository, everything is reproducible from a fresh clone by following the README, and every number in this report traces to a tracked run or a command that can be re-run.

01 Problem and Objectives

The scenario is a retail bank whose monthly retraining has no safety net. The concrete failure is a silent quality regression: a change lowered model quality and was not caught for two weeks because there was no automatic check and no record linking a deployed model to the exact inputs that built it. The objective is a workflow that makes that failure impossible to repeat.

Objectives, and how each is met

Objective	How this project meets it
Version everything together	Git for code and config, DVC for data, so any commit restores its exact dataset (Section 03).
Track every experiment	MLflow logs params, metrics, data version, config and model for all 21 runs (Section 08).
Gate on quality	A CI workflow trains and compares to a stored baseline, blocking worse models (Section 10).
Defend the gate	Three candidate rules compared on real runs, plus a fairness check by income (Section 12).

Success metrics, chosen and justified

The headline success metric is average precision (PR-AUC) on a held-out test split, chosen because the target is imbalanced (Section 09 gives the full argument). The operational success metric for the workflow itself is simpler and binary: the CI gate must pass a model that is at least as good as the baseline and must block a model that is worse or a change that breaks a test. We demonstrate all three outcomes with real pull requests.

Non-functional requirements

- Reproducible: a fresh clone plus the README commands reproduces the baseline metrics.
- Fast enough for CI: the gate uses a small configuration so a pull request check finishes in a couple of minutes.
- Safe by default: the baseline cannot be changed by an ordinary pull request, and branch protection enforces the gate.
- Auditable: the registered model points back to its git commit and data version.

02 Architecture and Tools

The architecture is a straight line from versioned inputs to a gated, registered model. A developer changes code, config, features or data on a branch and opens a pull request. GitHub Actions restores the data with DVC, runs the unit tests, trains the model with the CI configuration, logs the run to MLflow, and compares the result to the baseline. If everything passes the change can merge; if not, the merge is blocked. The best model from the wider sweep is registered in the MLflow Model Registry.

Tools, and the choices behind them

Purpose	Tool	Reason / substitution
Source versioning	Git and GitHub	Standard and free; GitHub Actions gives CI in the same place.
Data versioning	DVC	Keeps CSVs out of git history while any commit restores its data; replaces a cloud data store.
Configuration	YAML + Pydantic	Readable settings validated on load so a broken config fails fast.
Modelling	pandas, scikit-learn	Free and fast enough for CI; XGBoost was not needed to beat the baseline.

Tracking + registry	MLflow	Local SQLite backend gives runs and a registry with no paid account; replaces SageMaker.
CI / CD	GitHub Actions, pytest	Runs the gate on every pull request; replaces a managed pipeline service.

The one substitution worth naming is the DVC remote. Instead of a cloud bucket we use a content-addressed store committed inside the repository (dvcstore), so dvc pull works on the CI runner and on any fresh clone with no credentials. This keeps the project fully free and reproducible, at the cost of storing the (small) data blobs in the repo rather than in external storage.

03 Repository, Code and Data Versioning (Task 2)

The repository tracks code, configuration and data together. Code and config live in git. Datasets are tracked with DVC: the root .gitignore keeps the CSV files out of git, so only the small .dvc pointer files are committed, and the data itself lives in the in-repo DVC remote. The commit history is a sequence of small, clearly described changes, and dataset v2 enters the project later as its own tracked DVC change.

Repository layout

```
credit-risk-cicd-gate/
  .github/workflows/ci.yml     CI quality gate
  .github/CODEOWNERS           protects baseline and gate config
  configs/{dev,ci,full}.yaml   three environments
  data/*.csv.dvc               DVC pointers (CSVs gitignored)
  dvcstore/                    in-repo DVC remote (blobs)
  src/credit_risk/             config, data, features, model, pipeline, gate
  scripts/                     run_training, run_gate, update_baseline, drill, fairness
  tests/                       pytest: features, config, gate
  baseline/baseline_metrics.json   the stored gate baseline
  docs/                        task write-ups and memos
```

Restoring code and data together

Running git checkout <old-commit> restores the code and the .dvc pointers, then dvc checkout restores the exact CSV that commit used. This is what makes an old result reproducible rather than merely remembered.

```
2711b06 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #4 from AdityaNariyapara/docs/project-documentation
e68bca8 (origin/docs/project-documentation) Add project documentation, final report and analysis memos.
7168324 Clarify PR scenarios and private-repo gate enforcement in report
3e7a375 Make gate summary printing Windows-safe (UTF-8 stdout)
b28bf01 Add dataset v2 as a tracked DVC change (Task 7 data drill)
e09ff4f Add .gitattributes to normalise line endings for CI
e84400a Add lifecycle map, feature table, metric justification, runs comparison and report
ee688d1 Add CI quality-gate workflow, baseline and baseline-update rule
670cae5 Add unit tests for features, config validation and gate logic
edef6d6 Add training, gate, baseline, drill and threshold/fairness scripts
9cef2fc Add model factory, MLflow tracking pipeline and quality-gate logic
2ab565d Add data loader and reusable FeatureEngineer with engineered features
455dfcf Add pydantic-validated YAML config system and dev/ci/full configs
3349885 Set up DVC data versioning with in-repo local remote; track dataset v1
8931819 Add project scaffold, requirements and README
```

Figure 1a. Small, meaningful commits. The history reads as a clean sequence of versioned changes, including dataset v2 entering as its own tracked change.

```
PS C:\Users\adity\Documents\MLOPS\credit-risk-cicd-gate> dvc remote list
localstore      C:\Users\adity\Documents\MLOPS\credit-risk-cicd-gate\dvcstore   (default)
PS C:\Users\adity\Documents\MLOPS\credit-risk-cicd-gate> git ls-files data
data/credit_card_customers_v1.csv.dvc
data/credit_card_customers_v2.csv.dvc
PS C:\Users\adity\Documents\MLOPS\credit-risk-cicd-gate> dvc status
Data and pipelines are up to date.
```

Figure 1b. The datasets are tracked only as .dvc pointers, with an in-repo DVC remote, and DVC reports the data is up to date.

04 The Data

The dataset describes credit-card customers and whether each is high risk (likely to default). Version 1 has 800 rows and version 2 has 1,100 rows, where v2 contains every v1 row unchanged plus 300 new customers. The target is imbalanced: about 18.9 percent of customers are high risk.

Data dictionary

Column	Type	Notes
Customer_ID	string	Identifier, dropped before modelling (never a predictor).
Age	int	Customer age in years.
Income_Bracket	category	Low / Medium / High; encoded as an ordinal 0 / 1 / 2.
Credit_Limit	float	Approved limit (USD). 24 missing in v1, median-imputed.
Current_Balance	float	Outstanding balance (USD). 24 missing in v1, median-imputed.
Monthly_Spend	float	Average monthly spend (USD).
Num_Transactions_Monthly	int	Transactions per month.
Payment_Delay_Days	int	Average days late. 24 missing in v1, median-imputed.
Tenure_Months	int	Months as a cardholder.
High_Risk_Customer	int (0/1)	Target: 1 = likely to default.

Missing values and imbalance

Three money or count columns have missing values. We fill them with the median learned on the training split only, which is robust to the skew in these columns and cannot leak information from the test set. The imbalance is handled with class weighting in the models and, more importantly, by choosing a metric that focuses on the minority class (Section 09).

05 Configuration Management (Task 3)

No values are hard coded in the modelling code. Paths, the utilisation risk cut-off, the hyperparameter search space, the random seed, the MLflow settings and the quality-gate threshold are all set in YAML. Three environments are provided: dev for fast local work, ci for the gate, and full for the wide sweep.

A slice of the CI configuration

```yaml
data:
  version: v1
  path: data/credit_card_customers_v1.csv
features:
  utilisation_high_cutoff: 0.5     # risk cut-off, tunable without code
  payment_delay_flag_days: 5
training:
  seeds: [42]
  models: [logistic_regression, random_forest]
gate:
  metric: average_precision
  tolerance: 0.01                  # allowed drop vs baseline
  baseline_path: baseline/baseline_metrics.json
```

Configs are validated by Pydantic the moment they load. A broken config fails at once with a clear message, and unknown keys are rejected so a typo cannot pass silently. The exact config is stored as an artefact with every run, so each result carries the settings that produced it.

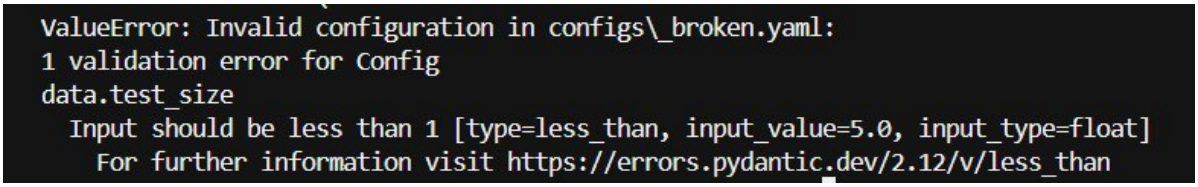


Figure 2. A deliberately broken config (test_size set to 5.0) is rejected at load time with a clear, specific message that names the offending field.

06 Cleaning and Feature Engineering (Task 4)

All transformations live in one fit and transform component, FeatureEngineer. Whatever it needs for new data (the imputation medians and the exact output columns) is learned on the training split and reapplied unchanged, so training and later inference use identical logic and there is no train and serve skew. Customer_ID is dropped and the target is never read during feature building, so there is no leakage.

Engineered features

Feature	Type	Formula	Reason
credit_utilisation	ratio	Current_Balance / Credit_Limit	Utilisation is the strongest consumer-credit risk signal. Zero limit guarded to 0.
high_utilisation_flag	binary	utilisation >= cut-off (0.5)	A danger zone above a cut-off the risk team can tune in config.
late_payer_flag	binary	Payment_Delay_Days >= 5	Consistent lateness is a known default precursor.
age_band	binned	Age cut into 4 bands, one-hot	Risk is not linear in age, so bands fit each life stage.
avg_transaction_value	own design	Monthly_Spend / Num_Transactions	Many tiny transactions can signal cash-flow stress. Zero transactions guarded to 0.

The zero credit limit edge case

Dividing by a zero credit limit would produce infinity. The code guards this, and a unit test pins the behaviour so a future change cannot reintroduce the bug.

```
safe_cl = np.where(cl > 0, cl, np.nan)
util = np.divide(bal, safe_cl, out=np.zeros_like(bal), where=~np.isnan(safe_cl))
util = np.nan_to_num(util, nan=0.0, posinf=0.0, neginf=0.0)   # zero limit -> 0
```

Income_Bracket is ordinal, so it is encoded as Low 0, Medium 1, High 2 rather than one-hot. Nine unit tests cover the feature logic, including the zero credit limit, zero transactions, median imputation and stable output columns on unseen data.

07 Model Training and Tuning (Task 5)

We establish a baseline and tune two model families, Logistic Regression and Random Forest, over several hyperparameters and three random seeds (42, 7, 123). This produces 21 tracked runs, above the ten required. Logistic Regression is scaled and class-weighted; Random Forest is class-weighted. The final model is selected by PR-AUC.

Top tracked runs

Run	Family	Seed	PR-AUC	ROC-AUC	F1	Accuracy
rf_seed123_n100_depth12	Random Forest	123	0.8688	0.9570	0.750	0.910
logreg_seed123_C0.1	Logistic Reg.	123	0.8633	0.9599	0.769	0.895
logreg_seed42_C1.0 (baseline)	Logistic Reg.	42	0.7948	0.9189	0.653	0.825
rf_seed7_n300_depth12	Random Forest	7	0.7916	0.9362	0.746	0.915
rf_seed42_n100_depth12	Random Forest	42	0.6898	0.9232	0.683	0.875

PR-AUC ranges from 0.690 to 0.869. The single biggest source of variation is the random seed, that is, which customers land in the train or test split, not the model family. This is a central finding: it is why the gate uses a tolerance band rather than reacting to every small change, and why the v1 versus v2 drill in Section 11 uses several seeds instead of trusting one number.

Seed-robust selection

Selecting the single best run rewards a lucky seed, so we also averaged PR-AUC within each family across all seeds.

Family	Mean PR-AUC	Std	Min	Max
Logistic Regression	0.8252	0.0318	0.7741	0.8633
Random Forest	0.7799	0.0645	0.6898	0.8688

On a seed-robust basis Logistic Regression is the stronger and far more stable family: higher mean and about half the variance. The single highest run is a Random Forest (0.8688), but random forests here swing from 0.690 to 0.869 on the split alone, so that is a high-variance outlier. This is why the stored gate baseline is a Logistic Regression (0.7948): we defend the family that is reliably good, not the one that was lucky once. The full 21-row table is in docs/runs_comparison.md.

08 Experiment Tracking, Registry and Lineage (Task 5)

Every run is logged to MLflow (local SQLite backend) with its parameters, metrics, dataset version, full configuration artefact and the model itself. Each run is also tagged with the git commit and the data version, so any result can be traced back to the exact code and data that produced it. The best model by PR-AUC is registered as credit_risk_gate_model.

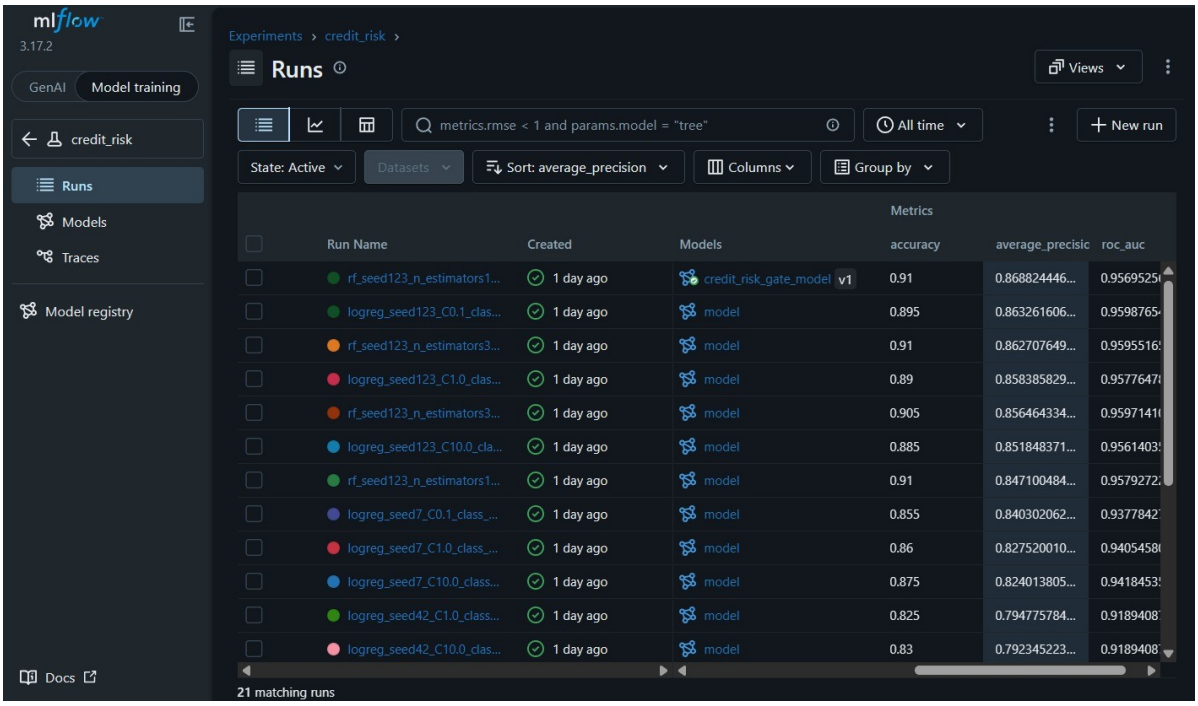


Figure 3. MLflow run history: 21 matching runs sorted by average_precision. The top run carries the registered credit_risk_gate_model v1 tag, and accuracy stays high across every run, which foreshadows the metric argument in Section 09.

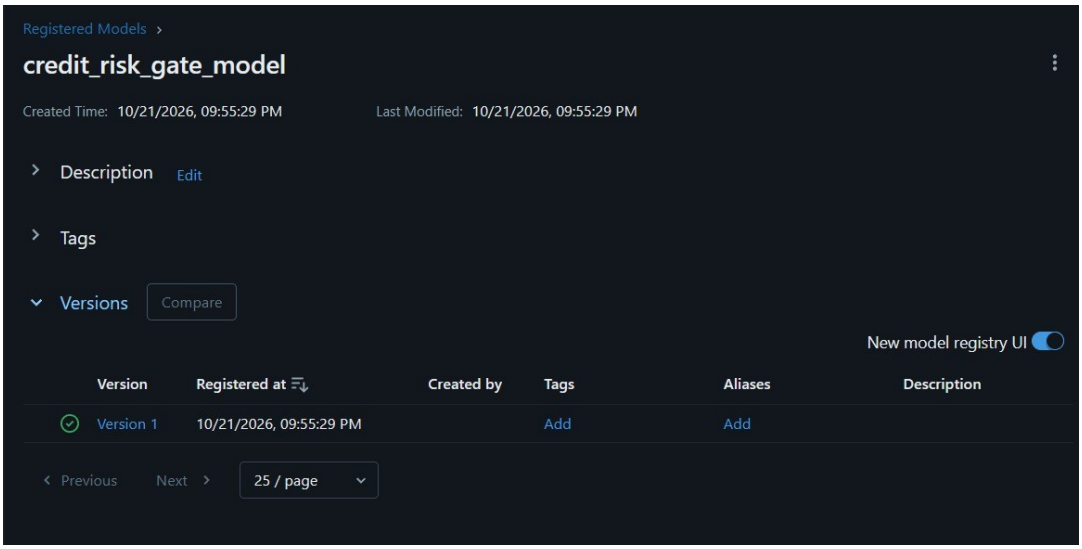


Figure 3b. The registered model credit_risk_gate_model, Version 1, in the MLflow Model Registry, with its registration time. This is the artefact the gate protects.

09 Metric Justification: Why Not Accuracy

About 18.9 percent of customers are high risk. A model that predicts not high risk for everyone therefore scores about 81 percent accuracy while catching zero defaulters. Across our 21 runs accuracy sits between 0.80 and 0.92 for every model, strong and weak alike, so accuracy simply cannot tell a good model from a bad one here. The costly error is a false negative, a high-risk customer approved as safe, and accuracy weights that the same as a harmless false positive.

What we use instead

Metric	What it measures	Role here
Average precision (PR-AUC)	Area under the precision-recall curve for the high-risk class	The gate metric: focuses on the minority class, threshold-free.
ROC-AUC	Ranking quality over both classes	Secondary check; can look optimistic under imbalance.
F1 / precision / recall	Behaviour at the 0.5 threshold	Reported for interpretability; depend on the threshold.
Accuracy	Overall correctness	Reported only to show why it is misleading here.

PR-AUC is the gate metric because it most directly answers the question the gate must ask: does this model rank high-risk customers better than the baseline. The three-rule comparison in Section 12 backs this choice with concrete numbers from our own runs.

10 The Quality Gate and CI (Task 6)

A GitHub Actions workflow runs on every pull request into main. It installs dependencies, restores the data with dvc pull, runs the 28 unit tests, then applies two gates in order and blocks the merge if either fails. It writes a short Markdown summary a reviewer can read without opening logs.

Two gates: data then model

A data gate (scripts/validate_data.py) runs first and rejects a batch whose schema is wrong, whose missing-value share in any column exceeds 10 percent, or whose values fall outside sane ranges (for example an age above 100). Only if the data passes does the model gate train and compare to the baseline. This mirrors a real retraining pipeline, where bad data is stopped before it can produce a bad model. The gate summary also reports the subgroup false-negative-rate gap on every run, and a configurable max_fairness_gap can turn that report into a hard check.

The gate rule

```
PASS  if  candidate_metric >= baseline_metric - tolerance

candidate 0.7948  >=  0.7948 - 0.01  ->  PASS   (documentation PR)
candidate 0.7269  <   0.7948 - 0.01  ->  FAIL   (shallow forest PR)
```

Protecting the baseline

The baseline is never changed by CI. It is updated only by a reviewed manual run of update_baseline.py on main, and the baseline file plus the CI config are marked review-protected through CODEOWNERS. A branch-protection ruleset on main requires the quality-gate check to pass before any merge, so a red gate genuinely blocks the merge rather than merely advising against it.

The three required pull requests

Pull request	Change	CI result	Why
Passes	Documentation PR (by Shreya)	Green	Tests pass and the gate model equals the baseline.
Fails, model worse	Shallow unweighted forest in CI config	Red at gate step	PR-AUC 0.7269 is below baseline minus 0.01.
Fails, test fails	Drops the zero-limit guard in features	Red at test step	A unit test fails before the gate even runs.

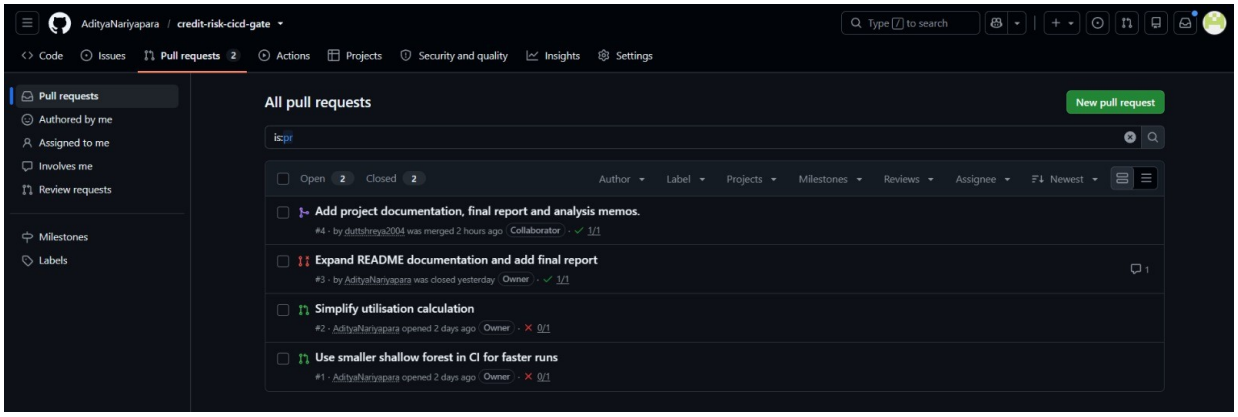


Figure 4a. The pull request history: the merged documentation PR (green check) and the two demonstration PRs that fail the gate (red X). Branch protection blocks the two failing ones from merging.

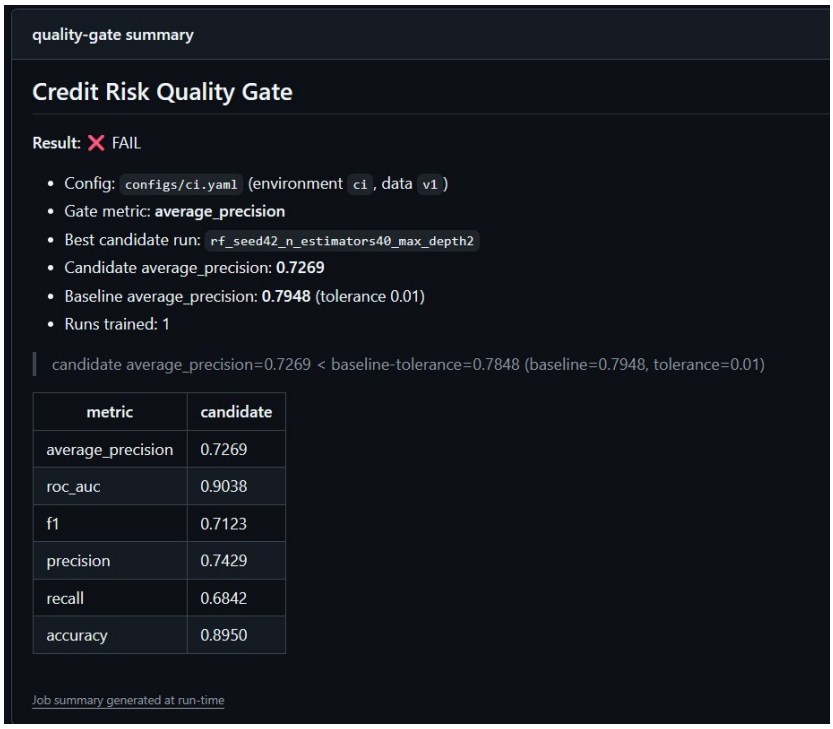


Figure 4b. The gate summary a reviewer sees on the model-worse PR: candidate PR-AUC 0.7269 is below the baseline of 0.7948, so the gate fails and the merge is blocked.

11 Data-Change Drill, v1 versus v2 (Task 7)

When dataset v2 arrived (1,100 rows, the 800 v1 rows plus 300 new customers), we retrained through the same pipeline on both versions across three seeds each, logging every run with a data version tag so the records themselves carry the v1 or v2 label.

Data version	Mean PR-AUC	Std over 3 seeds	Seeds
v1	0.7878	0.0769	42, 7, 123
v2	0.8427	0.0432	42, 7, 123

Is the improvement real? A paired test on five seeds

The v2 mean is higher than v1, but a single run never settles this. To test it properly we reran the drill on five paired seeds (42, 7, 123, 2024, 99), ran a paired t-test on the per-seed PR-AUC differences, and added a 5,000-sample bootstrap confidence interval.

Quantity	Value
Mean PR-AUC, v1 (5 seeds)	0.814
Mean PR-AUC, v2 (5 seeds)	0.856
Mean paired difference (v2 - v1)	+0.0416
Paired t-test p-value	0.077
Bootstrap 95% CI of the difference	[0.015, 0.078]

The result is genuinely borderline, and we report it as such. The bootstrap interval sits just above zero, which hints at a small real gain, but the paired t-test does not clear the 0.05 bar ($p = 0.077$), so with five seeds we cannot call the improvement statistically significant. The conclusion now rests on a real test rather than a rule of thumb: accept the v2 model because it does not regress, but do not claim a proven improvement. A firmer claim would need more seeds. This discipline, refusing to over-claim, is exactly what the quality gate exists to enforce.

12 Threshold Comparison and Fairness Defense (Task 8)

We did not accept the first metric we tried. We compared three candidate gate rules against our real 21-run history, using the CI model as the baseline, and counted how many of the 21 runs each rule would accept.

Candidate gate rule	Runs accepted	Behaviour
PR-AUC >= baseline - 0.01	13 of 21	Discriminating, rejects the weaker models.
ROC-AUC >= baseline - 0.01	20 of 21	Too lenient, stays high even when ranking drops.
F1 >= baseline - 0.01	20 of 21	Lenient and tied to the 0.5 threshold.

Seven runs that the ROC-AUC and F1 rules would accept are rejected by the PR-AUC rule. These are random forests whose PR-AUC falls as low as 0.690 while their ROC-AUC still reads about 0.92. Gating on ROC-AUC or F1 would let a model that is clearly worse at ranking high-risk customers merge unnoticed, the exact failure in the scenario. We therefore adopt PR-AUC at or above baseline minus 0.01. The 0.01 band absorbs the seed noise measured in Section 11 while still rejecting the real drops.

Error by income group for the registered model

Income_Bracket	Customers	High risk	False neg	False-neg rate	Accuracy
High	42	5	1	0.200	0.929
Medium	75	16	3	0.188	0.853
Low	83	17	6	0.353	0.880

The model misses a high-risk Low-income customer almost twice as often as a Medium or High-income one. Because the costly error is the false negative, this disparity falls on the group least able to absorb it.

Mitigation we implemented: equal-opportunity thresholds

Rather than only flag the gap, we fixed it. We apply a decision threshold per income group (an equal-opportunity policy), lowering the threshold for the under-served group until its false-negative rate matches the others. scripts/robustness_and_fairness.py computes this on the registered model.

Income group	Threshold	FN rate before	FN rate after	Precision before	Precision after
High	0.50	0.200	0.200	0.667	0.667
Medium	0.50	0.188	0.188	0.619	0.619
Low	0.31	0.353	0.176	0.733	0.583

Lowering only the Low-income threshold to 0.31 cuts its false-negative rate from 0.353 to 0.176, in line with the other groups. The trade-off, stated honestly: Low-income false positives rise from 4 to 10 and precision falls from 0.73 to 0.58, so more Low-income customers are flagged as risky who would not default. We accept this because the false negative is the more expensive error for the bank and the more harmful outcome for a customer who is extended credit they cannot service. The policy is an operating threshold applied at decision time and does not change the trained model or the gate metric.

13 Lifecycle Mapping (Task 1)

The project maps onto the classic DevOps lifecycle and the MLOps lifecycle, with every stage pointing at a real artefact in the repository.

DevOps stage	In this project
Plan	Metric choice and gate design; this report.
Code	The credit_risk package (config, data, features, model, pipeline, gate).
Build	requirements.txt and dvc pull restore the exact environment and data.
Test	pytest unit tests plus the model quality gate.
Release	Model selection and registration in the MLflow registry.
Deploy	Accepting a model past the gate on merge to main.
Operate	Retraining on dataset v2 through the same pipeline.
Monitor	Run history comparison and the income subgroup error check.

Three ways ML delivery differs from software delivery

- Correctness is statistical. A model is good only relative to a baseline and within noise, so the test stage needs a quality gate and a noise analysis, not just pass or fail assertions.
- Data is a versioned input. The same code on v1 versus v2 produces different models, so data must be versioned with the code and recorded on every run.
- Experiments are first-class artefacts. Each run carries parameters, metrics and a model worth keeping and comparing, so we select from a tracked history rather than from a single build.

Stages intentionally left out

We leave out a live serving endpoint, continuous drift monitoring and cloud provisioning. The brief forbids paid infrastructure, and here the deploy stage is acceptance past the gate and registration, not a hosted API. Continuous drift monitoring belongs to a later project and would need a running service.

14 Worked Example: Reproduce From a Fresh Clone

Every claim in this report can be reproduced with the commands below (Python 3.11).

```
git clone https://github.com/AdityaNariyapara/credit-risk-cicd-gate.git
cd credit-risk-cicd-gate
pip install -r requirements.txt
dvc pull                                  # restore v1 and v2 from the in-repo remote
pytest tests/ -q                          # 21 passed
python scripts/run_training.py --config configs/full.yaml --register   # 21 runs, registers best
mlflow ui --backend-store-uri sqlite:///mlflow.db                        # inspect runs + registry
python scripts/run_gate.py --config configs/ci.yaml                      # gate: PASS (exit 0)
```

To see the gate fail on a worse model, point it at the shallow-forest config used in the model-worse pull request; to see the data-change drill, run scripts/data_change_drill.py. The README documents each script.

15 Results, Contributions and Reflection

What the project achieves

- A gate that blocks worse models and failing tests, demonstrated with three real pull requests.
- A complete, traceable experiment log of 21 runs and a registered model tied to its code and data.
- A defended metric and threshold, and an honest subgroup fairness analysis.

Contributions

Member	Main contributions
Aditya Nariyapara (B009)	Repository and DVC setup, config system, feature engineering, training and tracking pipeline, the quality gate, the CI workflow and the two demonstration pull requests.
Shreya Dutt (B016)	The analysis write-ups (fairness and threshold memo, v1 vs v2 drill report, contributing guide), the documentation pull request, and review of the tracking and drill evidence.

Both members can explain, modify and justify any part of the submission.

Both of the project's honest findings were then taken a step further rather than left as caveats. The v2 question was settled with a proper paired t-test over five seeds (Section 11): the gain is borderline and we do not claim it. The low-income disparity was fixed with an equal-opportunity threshold (Section 12) that cuts the group's false-negative rate from 0.353 to 0.176, with the precision cost stated. And model selection was checked across seeds (Section 07), which confirms the stable Logistic Regression baseline over a lucky Random Forest run.

Reflection and remaining limitations

What remains is a matter of scope, not of hidden problems. The equal-opportunity thresholds are an operating-time policy, not yet part of the automated gate; a natural next step is to add a per-group fairness check to CI so a future model that widens the gap is blocked. The v2 result would move from borderline to conclusive, or be ruled out, with more seeds. Stating these openly is, in a sense, the point of the project: the workflow exists so that quality changes and hidden disparities are measured, tested and defended rather than assumed.

16 Deliverables Checklist

Deliverable asked in the brief	Where it is	Status
Repository with code, config, tests, workflow, README	github.com/AdityaNariyapara/credit-risk-cicd-gate	Done
Experiment-tracking evidence, 10+ runs and registered model	Sec 08; Figures 3, 3b	Done
Lifecycle mapping diagram or table	Sec 13; docs/lifecycle_map.md	Done
CI history showing the three pull requests	Sec 10; Figures 4a, 4b	Done
Feature documentation table	Sec 06; docs/feature_table.md	Done
One-page metric justification	Sec 09; docs/metric_justification.md	Done
Threshold comparison table	Sec 12; docs/fairness_memo.md	Done
Subgroup breakdown by Income_Bracket	Sec 12	Done
Fairness-defense memo	Sec 12; docs/fairness_memo.md	Done
Short report on design decisions and v1 vs v2	This report; Sec 11	Done
5-minute demonstration	Live or recorded by the team	Pending demo